Projet 1 – C’Hokayy

Delcorte Maxime

Morsiaux Thibaut

Vanni Thibault

Champs timestamp

- Valeur: Instant d’envoi du segment.

```
timestamp = int(time.time() * 500) & 0xffffffff # Masque 32-bit
```

- Le terme x500 permet de distinguer les segments qui sont envoyés à moins de 2 ms d’intervalle (2 ms car 1/500)

- Lors du décodage, si deux segments ont le même timestamp et seqnum, alors on sait que c’est un doublon.

Timeout

- Du coté server: RTO = 500ms car, sur base de nos tests, nous obtenons un RTT égal à environs 60ms, donc le RTO devrait être égal à RTT + 4*std_dev(RTT) ≅ 240 ms. Nous avons donc choisi de prendre une marge de sécurité égale à 2x le RTO.

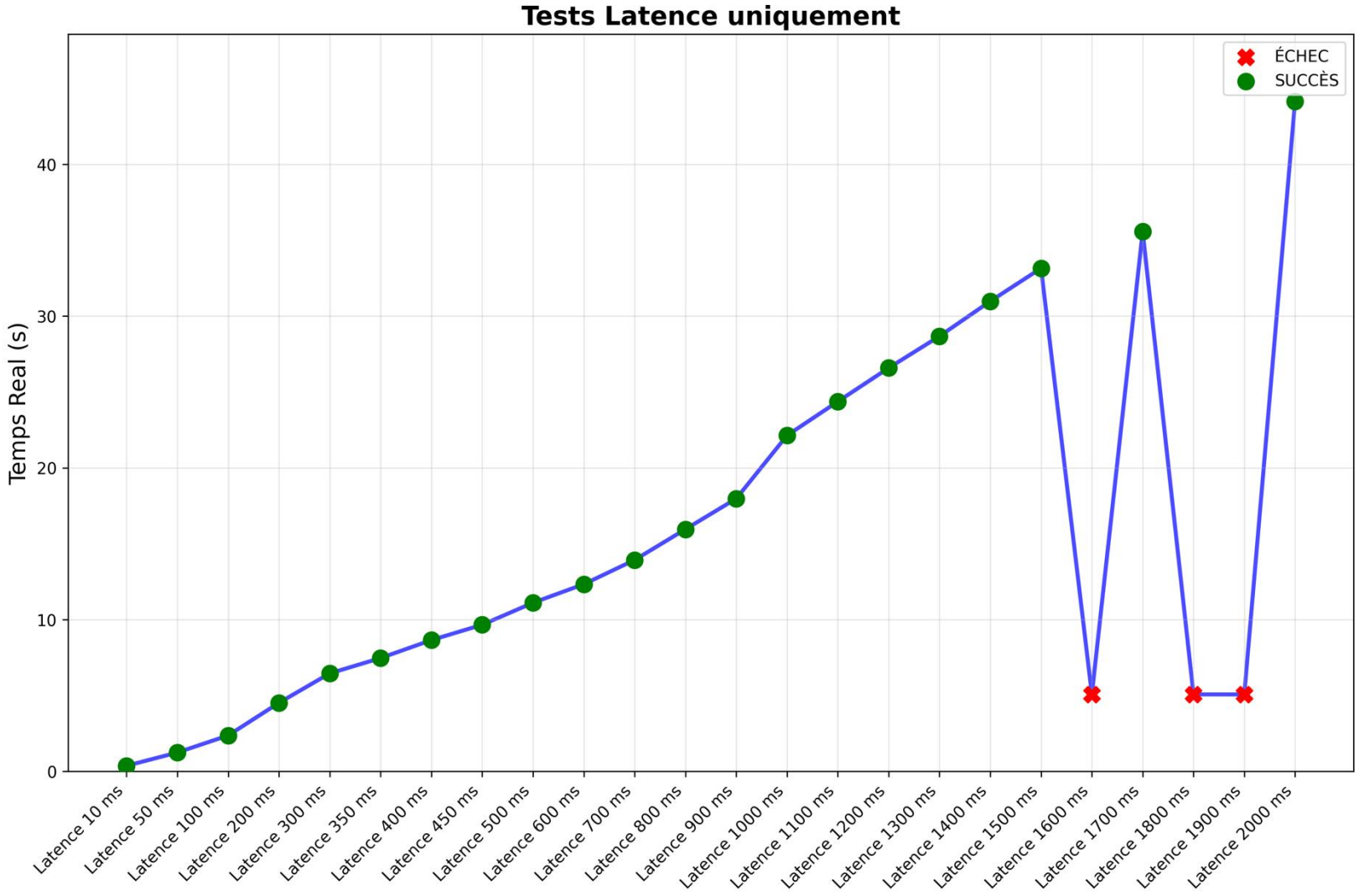
- Du coté du client: RTO = 5000ms car, dans les spécifications du projet il nous est indiqué que « la latence du réseau varie dans l'intervalle [0, 2000] ms. Donc pour un aller-retour des paquets il peut y avoir une latence maximale de 4000ms. Nous prenons donc une sécurité pour le RTO.

Partie critique

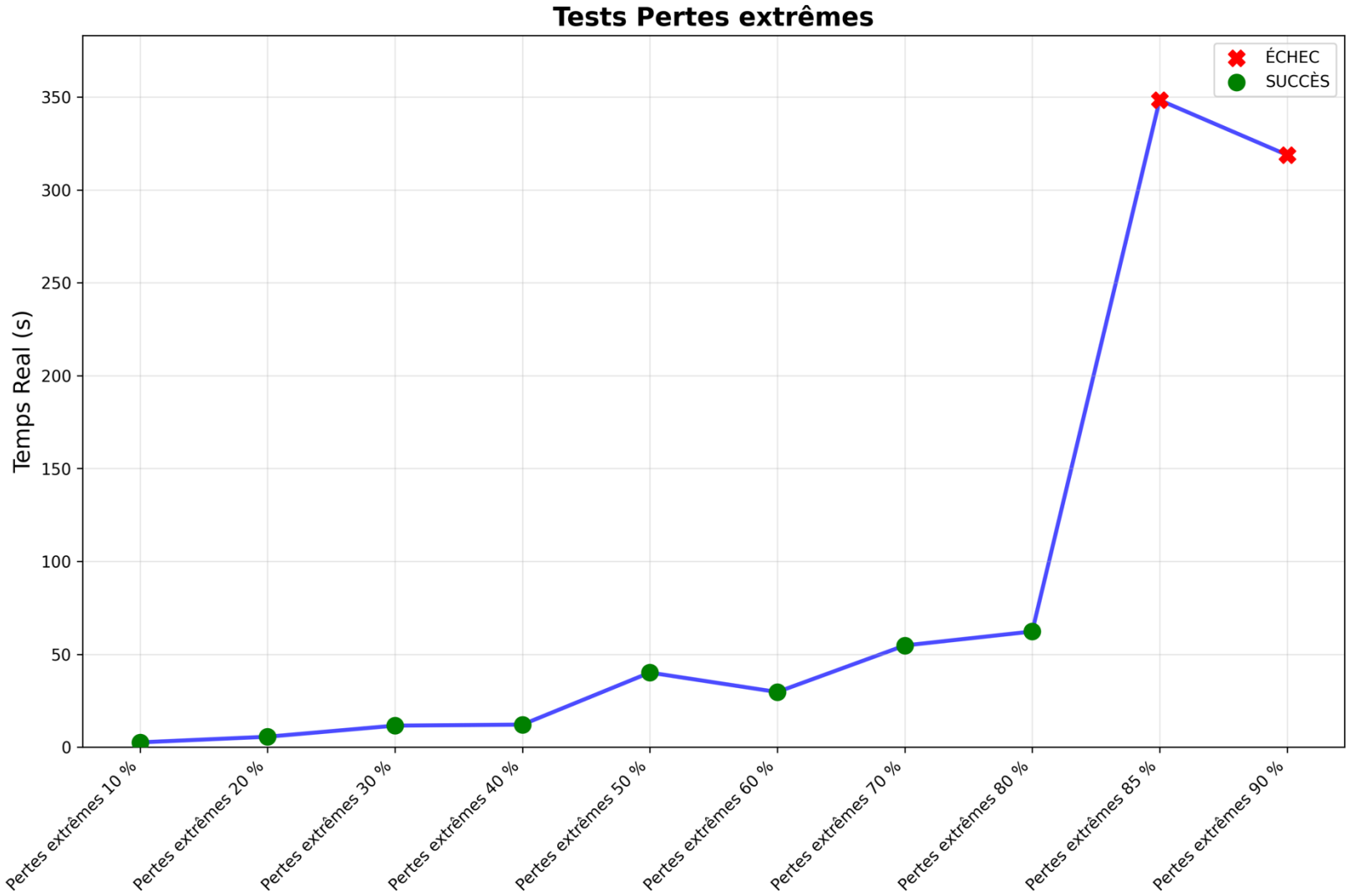
- Nous avons plusieurs soucis de lenteurs. Par exemple cette ligne:
  - if next_to_send in acked_indices:

  Où nous vérifions que le paquet a déjà été envoyé. C’est précisément le fait de contrôler dans une liste pour chacun des paquets qui est lent.
- Nous pourrions également diminuer le RTO du coté server.

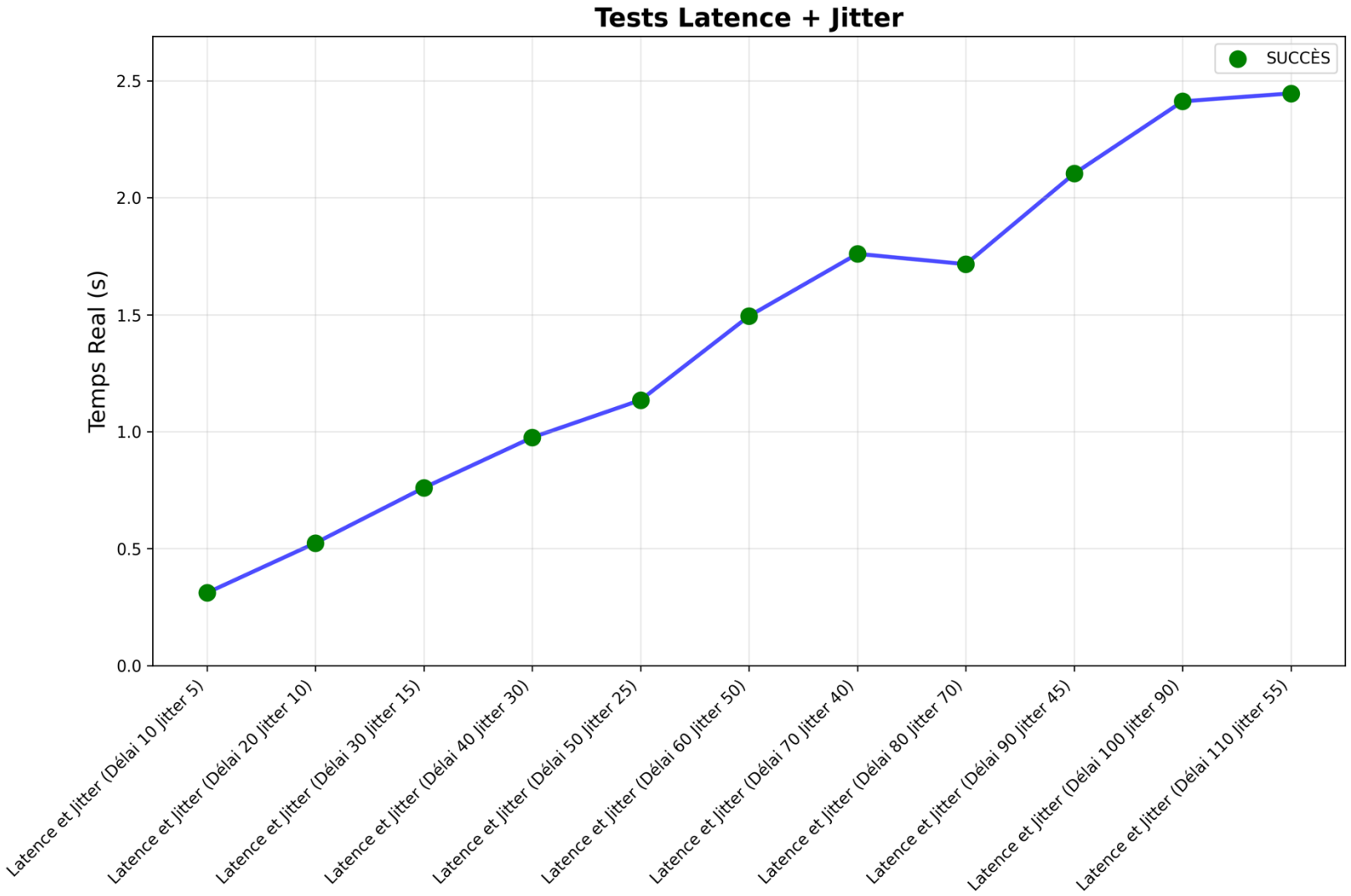
Performances - Latence



Performances - Pertes



Performances – Latence + Jitter



Stratégie de tests

- Nous avons également testé la robustesse de notre code en y appliquant certains cas. Pour chaque cas nous avons testé plusieurs valeurs (faibles, moyennes, élevées)
  - Nous avons d'abord tenté différentes latences.
  - Ensuite différents pourcentages de pertes des paquets
  - Enfin nous avons combiné une latence et un Jitter élevés
- Nous avons également testé notre travail par Link Simulator qui nous a permis de réaliser des tests sur un réseau parfait mais également des tests de pertes extrêmes, de latence et Jitter, de corruption de paquets, de troncation de paquets et enfin un test avec tout à la fois.